

Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

July 23, 2026

Contents

1	Thesis Plan	1
1.1	Plan Inspiration	1
1.2	Working Title	1
1.3	Central Thesis Question	1
1.4	Research Questions	1
1.5	Expected Contributions	1
1.6	Chapter Outline	2
1.7	Appendix Plan	4
1.8	Immediate Writing Order	4
1.9	Open Decisions	4
2	Methods	5
2.1	Homogeneous Graph Representation	5
2.1.1	Busbar nodes	5
2.1.2	Candidate line edges and dynamic topology	6
2.1.3	Physical scaling and running normalization	7
2.1.4	Additional relation edges	7
2.1.5	Local actor graphs and the global state graph	8
2.1.6	Graph encoder	8
2.2	Heterogeneous Graph Representation	9
2.2.1	End-to-end architecture	9
2.2.2	Typed nodes and features	9
2.2.3	Typed relations	10
2.2.4	Local heterogeneous graphs	11
2.3	Heterogeneous Graph with Transmission-Line Nodes	11
2.3.1	Node allocation and features	11
2.3.2	Endpoint-specific typed relations	12
2.3.3	Message-passing path through a line node	13
2.3.4	Local heterogeneous-line graphs	13
2.3.5	Architectural comparison of the two heterogeneous schemas	13
2.4	Substation Summary Nodes	14
2.5	Virtual-Node Graph Readout	15
2.5.1	Augmented graph	15
2.5.2	Learned summary instead of terminal pooling	15
2.5.3	Relation attributes and encoder compatibility	16
2.6	Graph Actor and Critic Configuration	16
3	Experimental Results	17
3.1	How to Read This Chapter	17
3.2	Evaluation Protocol	17
3.3	Experiment Index	17
3.4	IEEE 14-Bus Experiments	18
3.4.1	A0 Core Reruns	18
3.4.2	Initial Do-Nothing Bias	20
3.4.3	GINE Encoder Reruns	21
3.4.4	Heavy GINE	24
3.5	WCCI Action-Space Reduction	28

3.6	Decentralized Sparse Intervention Control	28
3.6.1	Evaluation-Time Rho Heuristics	29
3.6.2	Intervention-Gated Actor	29
3.6.3	Fixed Sparse-Action Penalty	30
3.6.4	Adaptive Intervention Budget	30
3.7	Current Takeaway	32
A	GINE and GAT Graph-Actor Architectures	34
A.1	Shared input and encoder pipeline	34
A.2	GINE message passing	34
A.3	GAT message passing	35
A.4	Pooling and graph output	35
A.5	Agent-specific action head	36
A.6	Exact node inputs by representation	36
A.7	Exact edge inputs by representation	36
B	Full-Test Checkpoint Registry	38

Chapter 1

Thesis Plan

1.1 Plan Inspiration

This plan follows the structure of the reference thesis in `mAI2024deMo1B.pdf`: start from the power-grid motivation and research questions, introduce the scientific background, describe the environment before the algorithms, then separate methods from results and close by answering the research questions. The adapted structure below is tuned to this repository: Grid2Op topology control, decentralized MAPPO, graph encoders, rerun ablations, and sparse intervention mechanisms.

1.2 Working Title

Decentralized Multi-Agent Reinforcement Learning for Sparse Power Grid Topology Control

1.3 Central Thesis Question

How can decentralized multi-agent reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces and avoiding unnecessary interventions?

1.4 Research Questions

1. How should the Grid2Op topology-control problem be formulated as a decentralized multi-agent decision problem?
2. Which architectural choices improve survival and generalization: flat MLP policies, graph neural network encoders, shared actor weights, critic design, or optimized critic updates?
3. How do initialization and training choices, especially the initial do-nothing bias and reward normalization, affect exploration and learned behavior?
4. Can sparse-control mechanisms, such as rho-based overrides, intervention gates, fixed penalties, or adaptive intervention budgets, reduce unnecessary topology actions without sacrificing episodic survival?

1.5 Expected Contributions

- A clear decentralized formulation of topology optimization in Grid2Op.
- A controlled comparison of MAPPO baselines, GNN encoders, critic variants, and initialization choices.
- An ablation study of sparse intervention mechanisms for more operator-like control.
- A reproducible experiment and analysis pipeline based on W&B histories, cached metrics, notebooks, and final evaluation scripts.

1.6 Chapter Outline

Chapter 1: Introduction

Goal. Motivate topology control as a low-cost but difficult operational tool for power-grid reliability.

Planned content.

- Energy-transition context: higher demand, renewable uncertainty, and stressed transmission networks.
- Topology control as an alternative or complement to redispatching and grid expansion.
- Why the problem is hard: sequential control, physical constraints, rare failures, and combinatorial topology actions.
- Why decentralized MARL is a natural fit: local substations, regional decision making, reduced per-agent action spaces, and scalability.
- Research questions, contributions, and thesis outline.

Chapter 2: Background and Related Work

Goal. Give the reader enough power-systems and RL background to understand the design choices.

Planned content.

- Power-grid operation: substations, busbars, line loading, overloads, contingencies, and remedial actions.
- Grid2Op and the Learning to Run a Power Network setting.
- Reinforcement learning basics: MDPs, policy gradients, PPO, and MAPPO.
- Multi-agent reinforcement learning: decentralized execution, centralized training, credit assignment, and non-stationarity.
- Graph neural networks for grid-structured observations.
- Safe, sparse, and constrained RL ideas relevant to intervention budgets.

Chapter 3: Environment and Problem Formulation

Goal. Define exactly what the agents observe, choose, optimize, and are evaluated on.

Planned content.

- Grid2Op environment used in the repository and the selected chronics.
- Multi-agent decomposition: agent-region or agent-substation assignment.
- Observation spaces: local grid state, line loadings, topology state, and optional graph observations.
- Action spaces: local topology actions, action 0 as do nothing, legal action constraints, and action-space reduction.
- Reward and evaluation metrics: episodic survival, illegal action rate, action-0 fraction, non-idle fraction, and train/test chronic split.
- Reproducibility setup: seeds, W&B logging, cached histories, and deterministic evaluation.

Chapter 4: Methods

Goal. Present the implemented learning systems and the ablations.

Planned content.

- MAPPO baseline: actor, critic, rollout collection, PPO objective, and centralized training assumptions.
- Baseline MLP agents and the A0 family of reruns.
- Graph-encoder agents: bus graph construction, GINE/GAT/GCN variants, node and edge pre-encoders, graph readout, and flat-feature concatenation.
- Critic variants: GNN critic versus MLP critic, legacy versus optimized critic updates.

- Initialization and exploration choices: entropy coefficient, entropy decay, and initial do-nothing probability.
- Sparse-control mechanisms: evaluation-time rho heuristic, intervention gate, fixed sparse-action penalties, and adaptive intervention budgets.
- Experimental protocol for ablations: controlled seeds, fixed evaluation cadence, and fair baseline comparisons.

Chapter 5: Experiments and Results

Goal. Show which design choices matter and what behaviors the agents learn.

Planned content.

- Main baseline performance: training stability and test survival.
- GNN architecture ablations: concat-flat, actor sharing, GINE versus GAT or weighted GCN, light versus full encoders, and critic design.
- A0 rerun ablations: optimized critic updates, smaller MLP, reward normalization, and do-nothing initialization bias.
- Sparse-control ablations: fixed penalty, global-safe weighting, local-safe adaptive budget, rho-threshold sensitivity, and budget target sensitivity.
- Behavioral analysis: how often agents act, whether interventions are local, and whether high survival comes with excessive topology churn.
- Generalization: seen versus unseen chronics and robustness across seeds.

Figures and tables to prepare.

- Table of run families and what each ablation changes.
- Survival curves for baseline, GNN, A0, and sparse-control comparisons.
- Action-frequency plots: action 0 fraction, any non-idle action, and multi-agent simultaneous interventions.
- Summary table with final survival, peak survival, and action sparsity.

Chapter 6: Discussion

Goal. Interpret the results rather than just restating them.

Planned content.

- Answer each research question directly.
- Explain which components appear essential, useful, or unnecessary.
- Discuss the tradeoff between survival and sparse interventions.
- Relate learned behavior to operator-like control: act rarely, act locally, and act when the grid is unsafe.
- Discuss limits: simulator scope, incomplete operational constraints, seed variance, training cost, and dependency on chosen chronics.

Chapter 7: Conclusion and Future Work

Goal. Close with the main thesis claim and next research directions.

Planned content.

- Short summary of the problem, approach, and core findings.
- Final conclusion about decentralized MARL for topology control.
- Future work: richer graph representations, larger grids, outage contingencies, constrained RL tuning, better coordination mechanisms, and operational validation.

1.7 Appendix Plan

- Appendix A: Environment details and action-space construction.
- Appendix B: Hyperparameters and training configuration tables.
- Appendix C: Full ablation run registry.
- Appendix D: Additional survival and behavior plots.
- Appendix E: Reproducibility notes for notebooks, cached W&B histories, and evaluation scripts.

1.8 Immediate Writing Order

1. Write Chapter 3 first, because it fixes notation and the experiment vocabulary.
2. Write Chapter 4 next, using the existing configuration READMEs as source notes for method variants.
3. Draft the Chapter 5 table of ablations before prose, so the results chapter has a stable spine.
4. Write Chapter 1 once the main conclusion is clearer.
5. Finish Chapters 6 and 7 after the final plots and summary metrics are frozen.

1.9 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Methods

2.1 Homogeneous Graph Representation

The homogeneous representation maps a Grid2Op observation to an undirected graph whose nodes correspond to busbars and whose edges describe possible transmission-line connections. Grid2Op exposes the instantaneous electrical state and topology through a structured observation [1]. The graph construction keeps the set of candidate nodes and edges fixed throughout an episode while using a dynamic edge mask to express the topology that is electrically active at time step t .

For agent a , the graph observation can be written as

$$\mathcal{G}_t^{(a)} = \left(\mathcal{V}^{(a)}, \mathcal{E}_{\text{cand}}^{(a)}, \mathbf{X}_t^{(a)}, \mathbf{Z}_t^{(a)}, \mathbf{m}_t^{(a)} \right), \quad (2.1)$$

where $\mathcal{V}^{(a)}$ and $\mathcal{E}_{\text{cand}}^{(a)}$ are static, $\mathbf{X}_t^{(a)}$ contains node features, $\mathbf{Z}_t^{(a)}$ contains edge features, and $\mathbf{m}_t^{(a)}$ indicates which candidate edges are active. This separates the physical equipment layout, which does not change during an episode, from the busbar assignment and line status, which can change after every topology action.

Figure 2.1 illustrates this baseline schema: busbars are the only nodes, while active transmission lines form the graph backbone.

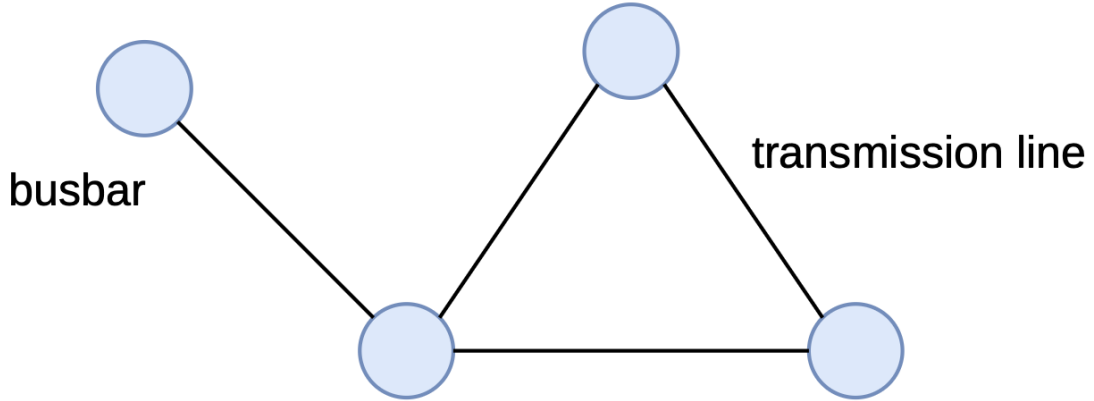


Figure 2.1: Busbar graph representation. Blue nodes denote busbars and black connections denote active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

2.1.1 Busbar nodes

Let S denote the number of substations and let B denote the maximum number of busbars per substation. A node is created for every substation–busbar pair, including busbars to which no asset is connected at the current time. A global node identifier is assigned as

$$i(s, b) = sB + b, \quad s \in \{0, \dots, S-1\}, \quad b \in \{0, \dots, B-1\}. \quad (2.2)$$

The complete graph therefore contains SB nodes. Grid2Op uses one-based busbar identifiers in its topology vector; these identifiers are converted to the zero-based index b in Equation 2.2.

Table 2.1 lists the six node features. Generator and load quantities are assigned to the busbar specified by the current topology vector. Active powers are summed when several assets occupy the same busbar, whereas the corresponding angles are averaged. Disconnected assets do not contribute to any node. The substation cooldown is repeated on every busbar node of that substation.

Table 2.1: Features attached to each busbar node.

Feature	Definition
gen_p	Sum of active generator power connected to the busbar.
gen_theta	Mean voltage angle of generators connected to the busbar.
load_p	Sum of active load power connected to the busbar.
load_theta	Mean voltage angle of loads connected to the busbar.
time_before_cooldown_sub	Remaining substation cooldown.
domain_mask	One for a substation controlled by the agent and zero for a contextual neighbor.

This aggregation produces a compact representation but deliberately removes the identities and counts of individual generators and loads. Consequently, two collections of assets with identical aggregated values have the same node representation.

2.1.2 Candidate line edges and dynamic topology

Consider a physical line ℓ whose origin and extremity belong to substations $o(\ell)$ and $e(\ell)$. For every pair of possible endpoint busbars (b_o, b_e) , the graph reserves the two directed candidate edges

$$i(o(\ell), b_o) \rightarrow i(e(\ell), b_e) \quad \text{and} \quad i(e(\ell), b_e) \rightarrow i(o(\ell), b_o). \quad (2.3)$$

Thus, each physical line contributes $2B^2$ candidate directed edges. The two directions share the same line attributes. For the usual case $B = 2$, one line produces eight candidate edges even though at most two—one per direction—are electrically active at a given time.

The mask for a candidate endpoint assignment is

$$m_{\ell, b_o, b_e, t} = \mathbf{1}[\text{line } \ell \text{ is connected at } t] \mathbf{1}[b_{\ell, t}^{\text{or}} = b_o] \mathbf{1}[b_{\ell, t}^{\text{ex}} = b_e]. \quad (2.4)$$

If a line is disconnected, all of its candidate edges are masked. Otherwise, only the pair corresponding to its current origin and extremity busbars remains active. Inactive edge features are set to zero, and inactive edges are removed before message passing. Topology changes can therefore modify graph connectivity without rebuilding the candidate graph.

The dynamic attributes copied onto active physical-line edges are given in Table 2.2. The two maintenance features are present only for environments with scheduled maintenance.

Table 2.2: Features attached to physical-line edges.

Feature	Definition
line_status	Binary connection status of the physical line.
rho	Line loading relative to its thermal limit.
timestep_overflow	Number of consecutive time steps in overload.
time_before_cooldown_line	Remaining cooldown before another line action is legal.
time_next_maintenance	Time until scheduled maintenance, when available.
duration_next_maintenance	Duration of scheduled maintenance, when available.

For GINE and GAT, the edge embedding of a busbar-to-busbar transmission edge is therefore exactly the row of Table 2.2. Optional self and same-substation edges use the same feature width: all physical channels are zero except **rho**=1, which is a neutral structural value. The homogeneous schema does not append relation one-hot, so these optional relations are distinguished only by their endpoints and neutral physical attributes. Inactive candidate edges are removed before either encoder is called; their zero feature rows never generate messages.

2.1.3 Physical scaling and running normalization

Two independent transformations can be applied to the continuous node and edge features of every graph schema: deterministic physical scaling followed by optional running standardization. Let $x_{i,f,t}$ be feature f of node or edge i at time t . The complete pipeline is

$$x_{i,f,t}^{\text{scale}} = \begin{cases} x_{i,f,t}/s_f, & \text{if physical scaling is enabled,} \\ x_{i,f,t}, & \text{otherwise,} \end{cases} \quad (2.5)$$

$$\hat{x}_{i,f,t} = \begin{cases} \text{clip}\left(\frac{x_{i,f,t}^{\text{scale}} - \mu_{f,t}}{\sqrt{\sigma_{f,t}^2 + \epsilon}}, -c, c\right), & \text{if running normalization is enabled,} \\ x_{i,f,t}^{\text{scale}}, & \text{otherwise.} \end{cases} \quad (2.6)$$

Here s_f is a fixed physical reference and $\mu_{f,t}$ and $\sigma_{f,t}^2$ are running moments estimated during training. When both transformations are enabled, the moments describe x/s_f . Clipping applies only to running standardization; $c = 10$ by default and a non-positive value disables clipping.

Deterministic physical scaling

Physical scaling divides each quantity by a constant with the same unit and therefore preserves physical zero. Table 2.3 gives the exact constants. The power base may be specified explicitly; otherwise the largest installed generator rating is used, with a 100 MW fallback when the environment does not expose generator ratings.

Table 2.3: Deterministic physical scales used for continuous graph features.

Features	Scale s_f	Source
<code>gen_p</code> , <code>load_p</code> , <code>p</code>	Power base in MW	Configured value or maximum generator rating.
<code>gen_theta</code> , <code>load_theta</code> , <code>theta</code>	180 degrees	Fixed angular scale.
<code>time_before_cooldown_sub</code>	Maximum substation cooldown	Grid2Op environment parameter.
<code>time_before_cooldown_line</code>	Maximum line cooldown	Grid2Op environment parameter.
<code>timestep_overflow</code>	Allowed overflow duration	Grid2Op environment parameter.
Maintenance time and duration	Episode horizon	Grid2Op chronic horizon.

Running standardization

Running moments are estimated with Welford’s online algorithm. One normalizer is shared across agents and updated from the full state graph once per environment step. Heterogeneous graphs maintain separate node statistics for each node type. Edge statistics include only active physical-line relations; inactive candidates and structural, asset-attachment, same-substation, and virtual relations do not contribute.

The loading ratio `rho` is already dimensionless and is neither rescaled nor standardized. Preserving it is also necessary when GCN uses `rho` directly as a non-negative message weight. Binary connection states, node and edge masks, domain masks, type indicators, and relation one-hot attributes are also left unchanged. Only the continuous channels listed in Table 2.3 enter the running statistics. The running moments are frozen during evaluation.

2.1.4 Additional relation edges

In addition to physical-line relations, the graph may include a self-edge $v \rightarrow v$ for every node and a directed same-substation edge $i(s, b) \rightarrow i(s, b')$ for every $b \neq b'$. These relations remain active throughout the episode and use zero physical-line features except for the neutral value `rho`= 1. Same-substation edges permit message passing between alternative busbars but do not represent an electrical connection.

For a full graph containing S substations and L physical lines, the number of candidate directed edges is

$$|\mathcal{E}_{\text{cand}}| = 2B^2L \quad (2.7)$$

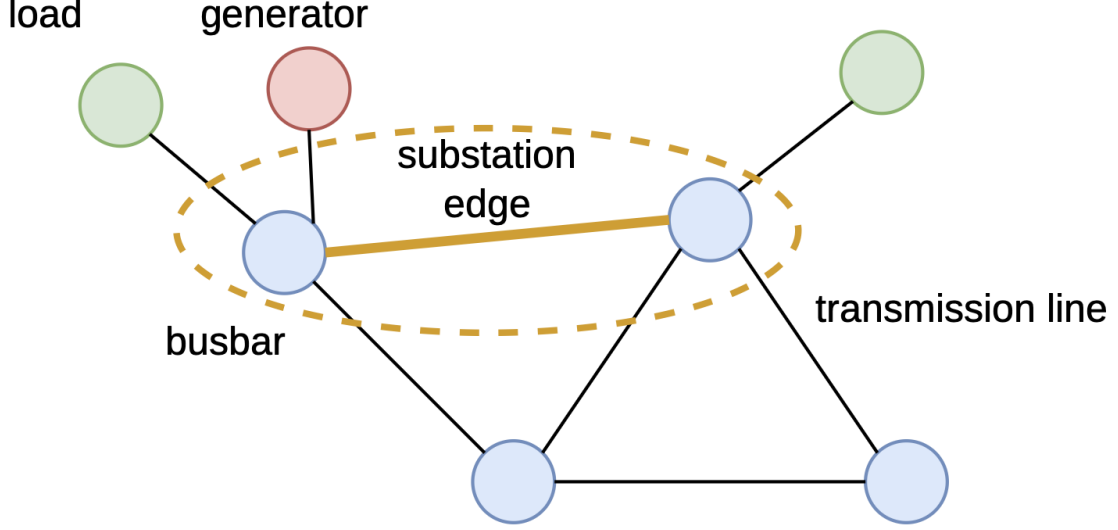


Figure 2.2: Direct same-substation busbar relation. The highlighted orange edge allows two busbars belonging to the same substation to exchange messages; it is a computational relation and not a physical transmission line.

for the standard representation, and

$$|\mathcal{E}_{\text{cand}}| = 2B^2L + SB + SB(B - 1) \quad (2.8)$$

when self and same-substation relations are both enabled.

2.1.5 Local actor graphs and the global state graph

Each agent controls a set of substations $\mathcal{C}_a$. Under decentralized observability, its local graph can be constructed in two ways. Without neighbor context, the node set contains only substations in $\mathcal{C}_a$, and the line set contains only lines whose two endpoints lie inside this region. With one-hop context enabled, the graph additionally contains every line touching $\mathcal{C}_a$ and the substations at the opposite endpoints of those lines. The `domain_mask` node feature and a separate controlled-node mask distinguish the controlled region from contextual neighbors. This second mask also permits graph readout to pool only the nodes on which the agent can act.

The full-grid state graph is available to the centralized critic during training, while each actor receives only its local graph. This follows the centralized-training, decentralized-execution structure of MAPPO [2].

2.1.6 Graph encoder

GINE and GAT receive every column of the edge-feature vector. The homogeneous graph provides only physical-line columns; the heterogeneous graphs append the relation one-hot columns defined in Tables 2.5 and 2.7. Node types are encoded by the explicit `is_*` node features.

GINE and GAT use the same high-level actor pipeline:

$$\begin{aligned} (\mathbf{X}, \mathbf{E}, \mathbf{m}) &\longrightarrow \text{active-edge filtering} \longrightarrow \text{node projection} \longrightarrow \mathbf{H}^{(0)}, \\ \mathbf{H}^{(0)} &\longrightarrow K \text{ message-passing layers} \longrightarrow \text{graph readout} \longrightarrow \text{action logits}. \end{aligned} \quad (2.9)$$

They differ only in the message-passing block. GINE inserts the projected edge vector into the content of every message. GAT uses the projected edge vector to compute the attention weight assigned to that message. Pooling, the graph output projection, and the action head are otherwise identical. The exact equations and architectural dimensions are given in Appendix A.

The busbar schema aggregates generator and load states and omits reactive power, voltage magnitudes, and directional endpoint flows. Empty busbars remain in the node set. The preprocessing in Section 2.1.3 changes feature values but not the graph schema.

2.2 Heterogeneous Graph Representation

The **heterogeneous** schema keeps busbars as the backbone and adds one node per generator and load. Transmission lines remain busbar-to-busbar edges. The node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}}, \quad (2.10)$$

with typed directed relations between these sets. Generator and load states are therefore preserved individually.

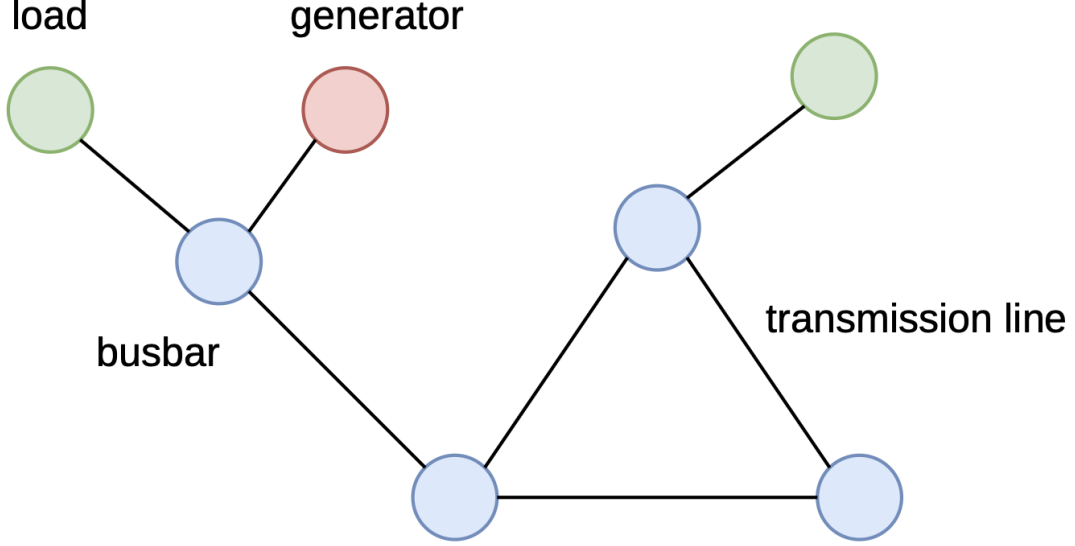


Figure 2.3: Heterogeneous equipment representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

2.2.1 End-to-end architecture

With bidirectional asset and line relations, information follows the path

$$\text{generator} \leftrightarrow \text{local busbar} \leftrightarrow \text{remote busbar} \leftrightarrow \text{load}. \quad (2.11)$$

One message-passing layer crosses one arrow. Generator or load information therefore reaches its local busbar after one layer and the adjacent remote busbar after two. With one-way asset relations, the reverse flow is absent. Mean readout includes all nodes in the local graph, including one-hop context.

2.2.2 Typed nodes and features

For a graph with S included substations, B busbars per substation, N_g included generators, and N_d included loads, the number of nodes is

$$|\mathcal{V}| = SB + N_g + N_d. \quad (2.12)$$

Busbar nodes are created exactly as in Section 2.1.1. In addition, each generator and load whose substation is present in the graph receives its own node. Asset nodes remain present when an asset is disconnected, so the node set stays fixed throughout an episode.

Every busbar, generator, and load node is represented by the same ordered eight-dimensional feature vector

$$\mathbf{x}_v = [\text{is_busbar}, \text{is_generator}, \text{is_load}, \text{p}, \text{theta}, \text{time_before_cooldown_sub}, \text{connected}, \text{domain_mask}]^T \in \mathbb{R}^8. \quad (2.13)$$

The width, column order, and meaning of this vector are identical for all node types; node type does not select a smaller feature vector. Table 2.4 shows the value placed in every shared column. A zero in the table means that the column is still present for that node type but is explicitly zero-filled.

Table 2.4: Values placed in the eight feature columns shared by every node in the direct-line heterogeneous graph. Disconnected assets retain their node row but have zero power and angle.

Shared feature column	Busbar node	Generator node	Load node
is_busbar	1	0	0
is_generator	0	1	0
is_load	0	0	1
p	0	gen_p if connected; 0 otherwise	load_p if connected; 0 otherwise
theta	0	gen_theta if connected; 0 otherwise	load_theta if connected; 0 otherwise
time_before_cooldown_subconnected	Substation value 1	0 1 connected; 0 disconnected	0 1 connected; 0 disconnected
domain_mask	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual

2.2.3 Typed relations

Every candidate directed edge is represented by the same ordered feature vector. Its physical block is

$$\mathbf{p}_{uv} = [\text{line_status}, \text{rho}, \text{timestep_overflow}, \text{time_before_cooldown_line}, (\text{time_next_maintenance}, \text{duration_next_maintenance})_{\text{optional}}]^T. \quad (2.14)$$

The two maintenance columns are appended only when scheduled maintenance is available. The relation block is the same seven-column vector for every edge:

$$\mathbf{r}_{uv} = [\text{relation_self}, \text{relation_physical_line}, \text{relation_same_substation_busbar}, \text{relation_generator_to_busbar}, \text{relation_busbar_to_generator}, \text{relation_load_to_busbar}, \text{relation_busbar_to_load}]^T. \quad (2.15)$$

The complete common edge vector is therefore

$$\mathbf{e}_{uv} = [\mathbf{p}_{uv}^T \mid \mathbf{r}_{uv}^T]^T \in \begin{cases} \mathbb{R}^{11}, & \text{without maintenance columns,} \\ \mathbb{R}^{13}, & \text{with maintenance columns.} \end{cases} \quad (2.16)$$

Its width, column order, and meaning do not change with the relation type. An active physical-line edge fills $\mathbf{p}_{uv}$ with its measured line state. Every active structural or asset-attachment edge instead uses

$$\mathbf{p}_{\text{neutral}} = [0, 1, 0, 0, (0, 0)_{\text{optional}}]^T, \quad (2.17)$$

so only the shared **rho** column is nonzero. In every active edge row, exactly one column of $\mathbf{r}_{uv}$ is one and the other six are zero.

Table 2.5 shows the values placed in these common blocks for every relation. An inactive candidate retains the same vector width, but its entire row is zero-filled and **edge_mask**=0 removes it before GINE or GAT message passing.

Table 2.5: Values placed in the common edge-feature vector shared by every active directed relation in the direct-line heterogeneous graph. “Other six” refers to the remaining columns of the seven-column relation block.

Directed relation	Active rule	Shared physical block	Shared relation block
$v \rightarrow v$ (self)	Optional; always active when enabled	$\mathbf{P}_{\text{neutral}}$	<code>relation_self= 1</code> ; other six = 0
$i_o \rightarrow i_e$ and reverse (physical line)	Line online and candidate busbars match the current endpoints	Measured $\mathbf{p}_{uv}$	<code>relation_physical_line= 1</code> ; other six = 0
$i(s, b) \rightarrow i(s, b'), b \neq b'$	Optional; every ordered same-substation busbar pair	$\mathbf{P}_{\text{neutral}}$	<code>relation_same_substation_busbar= 1</code> ; other six = 0
$g \rightarrow i(s(g), b)$	Generator connected to candidate busbar b ; direction enabled	$\mathbf{P}_{\text{neutral}}$	<code>relation_generator_to_busbar= 1</code> ; other six = 0
$i(s(g), b) \rightarrow g$	Same mask; reverse direction enabled	$\mathbf{P}_{\text{neutral}}$	<code>relation_busbar_to_generator= 1</code> ; other six = 0
$d \rightarrow i(s(d), b)$	Load connected to candidate busbar b ; direction enabled	$\mathbf{P}_{\text{neutral}}$	<code>relation_load_to_busbar= 1</code> ; other six = 0
$i(s(d), b) \rightarrow d$	Same mask; reverse direction enabled	$\mathbf{P}_{\text{neutral}}$	<code>relation_busbar_to_load= 1</code> ; other six = 0

Generator and load directions are selected independently as `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. Physical-line and same-substation relations remain bidirectional.

Physical lines use Equation 2.3. For generator g at substation $s(g)$, the bidirectional candidates are

$$g \rightarrow i(s(g), b) \quad \text{and} \quad i(s(g), b) \rightarrow g, \quad b \in \{0, \dots, B-1\}. \quad (2.18)$$

One-way modes retain only one member of each pair. Loads use the same rule. A disconnected asset has no active attachment.

Let $d_g, d_d \in \{1, 2\}$ denote the selected number of directions for generator and load attachments. Without optional self and same-substation relations, the candidate edge count is

$$|\mathcal{E}_{\text{cand}}| = 2B^2L + B(d_gN_g + d_dN_d), \quad (2.19)$$

where L is the number of included physical lines. Enabling both optional relations adds $|\mathcal{V}|$ self-edges and $SB(B-1)$ directed same-substation busbar edges.

2.2.4 Local heterogeneous graphs

A local actor graph contains controlled substations plus optional one-hop neighbors. Nodes belonging to the controlled substations form the action domain, while the additional neighbors provide context. The state graph contains the full grid. Relative to the busbar graph, this schema adds $N_g + N_d$ nodes and between $B(N_g + N_d)$ and $2B(N_g + N_d)$ asset candidates, but preserves individual asset states.

2.3 Heterogeneous Graph with Transmission-Line Nodes

The `heterogeneous_line` schema replaces each direct physical-line edge with a transmission-line node between its endpoint busbars. Its node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}} \cup \mathcal{V}_{\text{line}}. \quad (2.20)$$

2.3.1 Node allocation and features

For S included substations, B busbars per substation, N_g generators, N_d loads, and L included transmission lines, the graph contains

$$|\mathcal{V}| = SB + N_g + N_d + L \quad (2.21)$$

nodes. All types share one feature matrix. Table 2.6 lists the populated columns; unlisted columns are zero.

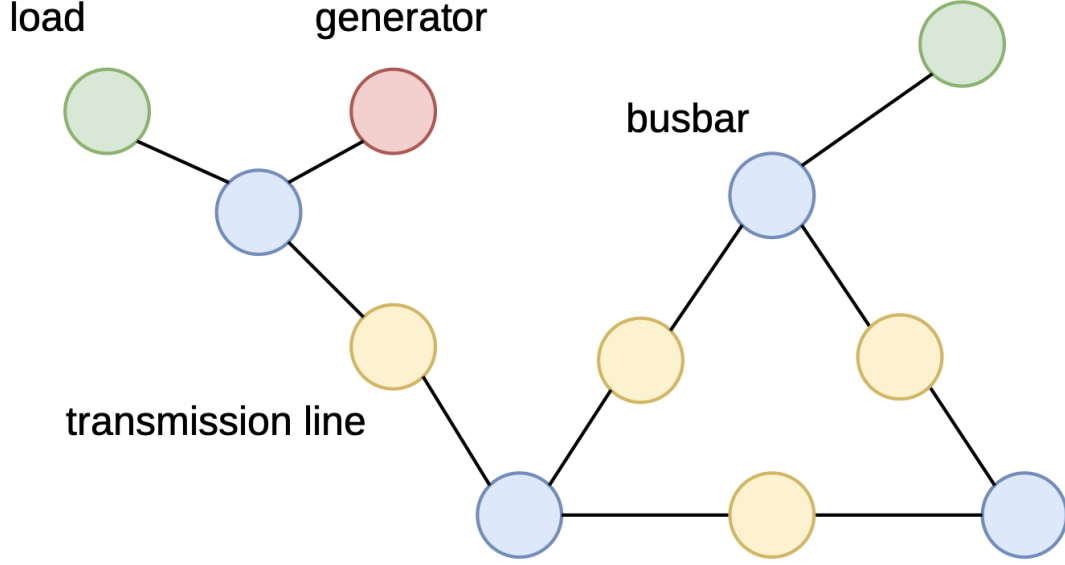


Figure 2.4: Heterogeneous representation with explicit transmission-line nodes. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain explicit terminal nodes.

Table 2.6: Node features by type in the explicit-line heterogeneous graph.

Node	Populated features	Meaning and zero rules
Busbar	<code>is_busbar=1</code> ; <code>time_before_cooldown_sub</code> ; <code>connected=1</code> ; <code>domain_mask</code>	Asset and line-state columns are zero.
Generator	<code>is_generator=1</code> ; <code>p=gen_p</code> ; <code>theta=gen_theta</code> ; <code>connected</code> ; <code>domain_mask</code>	Dynamic asset values are zero when disconnected.
Load	<code>is_load=1</code> ; <code>p=load_p</code> ; <code>theta=load_theta</code> ; <code>connected</code> ; <code>domain_mask</code>	Dynamic asset values are zero when disconnected.
Transmission line	<code>is_transmission_line=1</code> ; <code>line_status</code> ; <code>rho</code> ; <code>timestep_overflow</code> ; <code>time_before_cooldown_line</code> ; optional <code>time_next_maintenance</code> and <code>duration_next_maintenance</code> ; <code>connected</code> ; <code>domain_mask</code>	<code>p</code> , <code>theta</code> , and substation cooldown are zero.

A disconnected line remains allocated. Its `line_status` and `connected` are zero, its busbar attachments are masked, and cooldown or maintenance values remain visible.

2.3.2 Endpoint-specific typed relations

Let $o(\ell)$ and $e(\ell)$ denote the origin and extremity substations of line ℓ . For every busbar index b , the fixed candidate graph contains

$$i(o(\ell), b) \leftrightarrow \ell \quad \text{and} \quad i(e(\ell), b) \leftrightarrow \ell, \quad b \in \{0, \dots, B-1\}. \quad (2.22)$$

The line direction is `bidirectional`, `line_to_busbar`, or `busbar_to_line`. An online line has four active attachments in the bidirectional case and two in a one-way case. The mask selects the current origin and extremity busbars.

Here every edge feature is `[rho | relation one-hot]`. Line attachments carry the measured ρ_ℓ ; structural and asset edges use the neutral value one. Table 2.7 lists all ten relations.

Table 2.7: Edge relations and features in the explicit-line heterogeneous graph.

Directed relation	Active rule	Edge features
$v \rightarrow v$ (self)	Optional; always active when enabled	$[1 \mid \text{relation_self} = 1]$
$i(s, b) \rightarrow i(s, b'), b \neq b'$	Optional; every ordered same-substation pair	$[1 \mid \text{relation_same_substation_busbar} = 1]$
$i_o \rightarrow \ell$	Online line, current origin busbar, direction enabled	$[\rho_\ell \mid \text{relation_busbar_to_line_origin} = 1]$
$\ell \rightarrow i_o$	Same mask; reverse direction enabled	$[\rho_\ell \mid \text{relation_line_origin_to_busbar} = 1]$
$i_e \rightarrow \ell$	Online line, current extremity busbar, direction enabled	$[\rho_\ell \mid \text{relation_busbar_to_line_extremity} = 1]$
$\ell \rightarrow i_e$	Same mask; reverse direction enabled	$[\rho_\ell \mid \text{relation_line_extremity_to_busbar} = 1]$
$g \rightarrow i(s(g), b)$	Generator attached to b ; direction enabled	$[1 \mid \text{relation_generator_to_busbar} = 1]$
$i(s(g), b) \rightarrow g$	Same mask; reverse direction enabled	$[1 \mid \text{relation_busbar_to_generator} = 1]$
$d \rightarrow i(s(d), b)$	Load attached to b ; direction enabled	$[1 \mid \text{relation_load_to_busbar} = 1]$
$i(s(d), b) \rightarrow d$	Same mask; reverse direction enabled	$[1 \mid \text{relation_busbar_to_load} = 1]$

Origin, extremity, and direction have separate relation types. Generator and load attachments follow Equation 2.18; self and same-substation relations remain optional.

For this schema, GINE and GAT do not receive cooldown, overflow, maintenance, or status values on the attachment edges. Those quantities belong to the transmission-line node. An endpoint edge contains only the measured ρ and one endpoint-and-direction relation indicator.

2.3.3 Message-passing path through a line node

The direct-line graph connects endpoint busbars in one hop. The explicit-line graph uses

$$i_o \rightarrow \ell \rightarrow i_e, \quad (2.23)$$

so endpoint exchange requires two layers. The first layer forms

$$\mathbf{h}_\ell^{(1)} = f_\ell(\mathbf{h}_\ell^{(0)}, \mathbf{h}_{i_o}^{(0)}, \mathbf{h}_{i_e}^{(0)}), \quad (2.24)$$

and the second returns this joint state to the endpoints. Later layers enlarge the receptive field. Direction modes have the following effect:

- **bidirectional**: the line gathers both endpoints and sends the updated state back; endpoint exchange is possible;
- **line_to_busbar**: each busbar receives the line’s own state, but the endpoints do not exchange information;
- **busbar_to_line**: the line gathers both endpoints, but busbars do not receive the result; only a readout containing line nodes can use it.

The full line state is represented once on ℓ , rather than repeated on every candidate edge.

Let $d_\ell \in \{1, 2\}$ denote the number of selected line-attachment directions and retain d_g, d_d from the preceding section. Without the optional relations, the number of candidate directed edges is

$$|\mathcal{E}_{\text{cand}}| = 2Bd_\ell L + B(d_g N_g + d_d N_d). \quad (2.25)$$

For the common case $B = 2$, the line-attachment term has the same size as the $2B^2L$ direct busbar-to-busbar candidate set in Equation 2.19. For $B > 2$, explicit line nodes make this term grow linearly rather than quadratically with the number of busbars.

2.3.4 Local heterogeneous-line graphs

Local graphs use the same controlled-domain and one-hop rules. A line is marked controlled when either endpoint is controlled. The state graph contains the full grid. Relative to **heterogeneous**, this schema adds L nodes and one message-passing hop between adjacent busbars.

2.3.5 Architectural comparison of the two heterogeneous schemas

Table 2.8 summarizes the structural differences between the two heterogeneous schemas.

Table 2.8: Architectural comparison of the heterogeneous graph schemas.

Aspect	heterogeneous	heterogeneous_line
Node types	Busbar, generator, load	Busbar, generator, load, transmission line
Physical line	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Features of the line node
Endpoint path	One hop between busbars	Two hops through the line node
Minimum useful depth	One layer for adjacent busbar exchange; two for asset-to-remote-busbar flow	Two layers for adjacent busbar exchange through a line node
Hidden line state	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Graph size	Fewer nodes	Adds L line nodes

2.4 Substation Summary Nodes

An optional hierarchical augmentation introduces one learned node u_s for every included substation s . Let $\mathcal{B}(s)$ be its included busbar nodes. Summary edges are either **bidirectional** or **toward_summary**. In the latter case, the augmented graph contains only

$$\mathcal{E}_{\text{sub}} = \{(i, u_s) \mid i \in \mathcal{B}(s)\}; \quad (2.26)$$

the bidirectional setting additionally includes every reverse edge (u_s, i) . Each substation node therefore gathers information from every busbar belonging to that substation. In the bidirectional case it can also redistribute its learned summary to those busbars in later message-passing layers. It is not connected directly to generators, loads, or transmission-line nodes; those elements influence it through their busbar attachments.

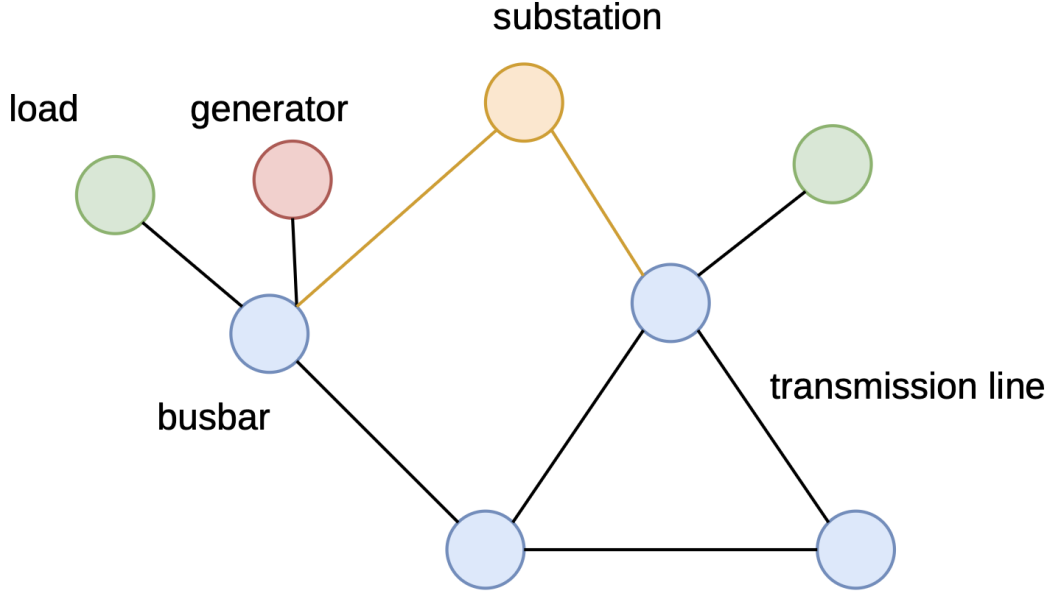


Figure 2.5: Learned substation-summary augmentation. The orange substation node receives messages from each of its blue busbar nodes and can optionally redistribute the resulting substation-level representation.

The initial substation-node states are learned embeddings indexed by the global substation identifiers. For S included substations and B busbars per substation, the augmentation adds S nodes and either SB directed edges for **toward_summary** or $2SB$ for **bidirectional**. The nodes participate in standard mean, sum, maximum, or attention readout. Controlled readout marks a substation node as controlled

whenever at least one of its busbars is controlled. The sparse graph transformer assigns the attachments a dedicated learned relation.

This augmentation differs from the same-substation edges in Section 2.1.4: it introduces a learned hub u_s instead of directly connecting pairs of busbars. It can be combined with the virtual-node readout.

2.5 Virtual-Node Graph Readout

A virtual-node readout replaces terminal mean, sum, maximum, or attention pooling with one learned graph-summary node. It can be combined with any of the three graph schemas.

2.5.1 Augmented graph

Let $v_\star$ denote the virtual node. Define the summary set $\mathcal{V}_{\text{sum}}$ as the included substation nodes when the augmentation from Section 2.4 is enabled, and as the included busbar nodes otherwise. The augmented graph is

$$\mathcal{V}^+ = \mathcal{V} \cup \{v_\star\}, \quad (2.27)$$

$$\mathcal{E}^+ = \mathcal{E} \cup \{(i, v_\star) \mid i \in \mathcal{V}_{\text{sum}}\}. \quad (2.28)$$

Equation 2.28 shows the `toward_summary` case; `bidirectional` additionally inserts $(v_\star, i)$ for every summary node. Thus the virtual node connects to all included busbars, or to all included substation-summary nodes when that augmentation is enabled. It has no direct edge to generator, load, or transmission-line nodes.

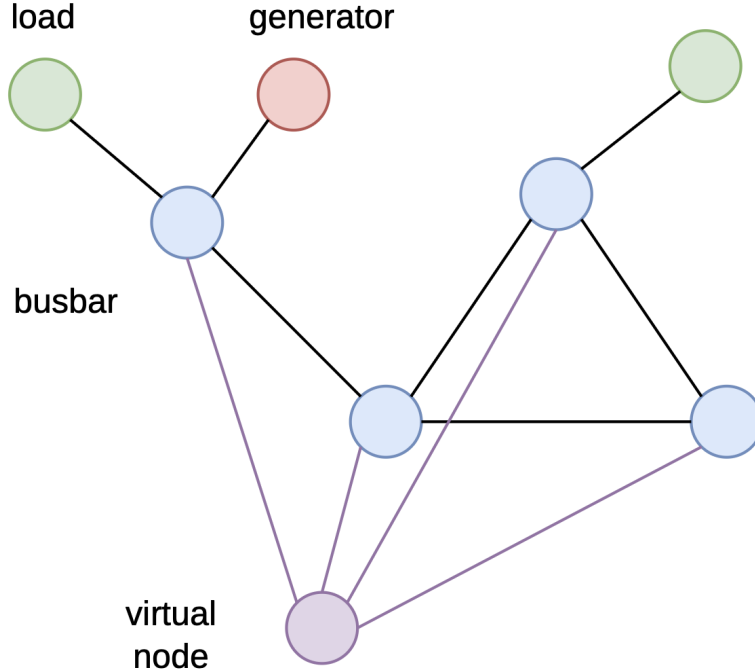


Figure 2.6: Virtual-node graph readout. The purple node receives messages from every included busbar and its final embedding replaces terminal node pooling; reverse messages are optional. When substation-summary nodes are enabled, the virtual node connects to those summaries instead.

2.5.2 Learned summary instead of terminal pooling

The initial virtual representation $\mathbf{h}_\star^{(0)}$ is learned. A message-passing layer jointly updates the physical and virtual nodes:

$$\left(\{\mathbf{h}_i^{(k+1)}\}_{i \in \mathcal{V}}, \mathbf{h}_\star^{(k+1)} \right) = \text{GNN}^{(k)} \left(\{\mathbf{h}_i^{(k)}\}_{i \in \mathcal{V}}, \mathbf{h}_\star^{(k)}, \mathcal{E}^+ \right). \quad (2.29)$$

With `bidirectional` edges the virtual state can also influence its neighbors in later layers; with `toward_summary` it only aggregates. After K layers, the graph embedding is

$$\mathbf{z}_G = \text{ReLU} \left(\mathbf{W}_o \mathbf{h}_*^{(K)} + \mathbf{b}_o \right), \quad (2.30)$$

rather than from $\text{Pool}(\{\mathbf{h}_i^{(K)}\}_{i \in \mathcal{V}})$. The output dimension is unchanged.

2.5.3 Relation attributes and encoder compatibility

Virtual connections are structural: their continuous edge attributes are zero, except for a neutral unit value in the `rho` position when that feature exists. The unit value ensures that a GCN configured to use `rho` as an edge weight does not suppress virtual messages. The sparse graph transformer allocates dedicated learned relations for substation attachments and virtual-node attachments, allowing its attention layers to distinguish hierarchy links from all physical and equipment relations.

Without substation nodes, a graph containing N_{bus} included busbars receives one virtual node and either N_{bus} or $2N_{\text{bus}}$ directed edges. With the hierarchy enabled, the virtual-node stage instead adds either S or $2S$ edges on top of the substation augmentation. After one layer the virtual node has aggregated its immediate busbar or substation neighbors.

2.6 Graph Actor and Critic Configuration

The actor uses two message-passing layers with 128 hidden features. LayerNorm is applied after the input node projection and after each message-passing layer. The graph readout produces a 128-dimensional embedding, followed by an agent-specific action head with two 128-unit hidden layers. The graph encoder and output projection are shared across agents; the action heads are separate because their action-set sizes may differ. The flat observation is not concatenated to the graph embedding, and learned global node-identifier embeddings are disabled. Local actors include one-hop neighboring context.

The centralized critic is a separate MLP over the normalized flat state and does not reuse the actor graph embedding. It is used only during centralized training. Graph-input normalization follows Section 2.1.3 and is independent of the critic’s flat-state normalization. The exact GINE and GAT updates, graph readout, output projection, and action-head equations are given in Appendix A.

Chapter 3

Experimental Results

3.1 How to Read This Chapter

This chapter is organized as a sequence of experiment blocks. The goal is to make the thesis easy to navigate while many tests are still being added. Each block follows the same structure:

1. **Question.** What uncertainty or design choice does this test address?
2. **Baseline and change.** Which run is the control, and what is changed in the comparison runs?
3. **Protocol.** Which data split, checkpoint, seeds, and metric are used?
4. **Implementation detail.** Which algorithmic, architectural, or mathematical change is being tested, when this is relevant?
5. **Evidence.** Which table and figure summarize the result?
6. **Interpretation.** What can be concluded, and what still needs to be verified?

Using the same block structure for every new experiment should make it possible to add future MSc-thesis tests without rewriting the whole results chapter.

3.2 Evaluation Protocol

The experiments use a train/test chronic split. The training set contains 80% of the chronics and is used to update the policy. The test set contains the remaining 20% and is used to estimate generalization during training and for final evaluation.

During training, evaluation rollouts are periodically run on both the train and test splits using 10 episodes each time. In the current rerun notebook, the main reported metric is **test episodic survival**, expressed as a percentage. Higher survival means that the agent keeps the grid alive for a larger fraction of the evaluation episodes. The final plots show seed-aggregated mean curves, faint individual seed curves, and the variability band.

The final evaluation currently uses the saved checkpoint with the best combined train/test survival. This selection rule should be reported explicitly because the test split participates in checkpoint selection; the values in this chapter are therefore best interpreted as controlled rerun comparisons, not as a final untouched test-set estimate.

3.3 Experiment Index

Table 3.1 gives a compact map of the experiment blocks included so far. New result sections should add one row to this index before adding detailed prose.

Table 3.1: Current result blocks and their role in the thesis.

Block	Main question	Primary evidence	Status
A0 core reruns	Which baseline training changes stabilize MAPPO?	Table 3.3, Figures 3.1 and 3.2	Drafted
Initial do-nothing bias	Does an initial action-0 prior help exploration?	Table 3.4, Figure 3.3	Drafted
GINE reruns	Do graph encoders improve survival over the A0 baseline?	Tables 3.5 and 3.6, Figure 3.4	Drafted
Heavy GINE	Does a larger GINE architecture help or overcomplicate optimization?	Table 3.7, Figure 3.5	Drafted
Sparse intervention control	Can decentralized agents intervene rarely, locally, and only when useful?	Tables 3.9 and 3.10	Design drafted

3.4 IEEE 14-Bus Experiments

The first result group focuses on the IEEE 14-bus topology-control task. The initial PPO/MAPPO baseline from the MARL2Grid framework was not satisfactory: the agents did not learn a robust policy and the survival rate remained low. The following experiments therefore test training and architecture changes that could make the baseline more stable.

3.4.1 A0 Core Reruns

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. The A0 family starts from the flat MLP MAPPO baseline and tests three main stabilization choices:

- increasing actor and critic capacity from two hidden layers to three hidden layers;
- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent.

Implementation detail. There is no new mathematical objective in this ablation. The implementation changes are practical training choices: network depth, reward normalization, critic-update scheduling, and the initialization of the action-0 probability. Table 3.2 records these settings so that the survival plots can be interpreted without repeatedly restating the run names.

Evidence. Table 3.3 reports final and peak survival for the core A0 ablations. Figure 3.1 shows each comparison against the `rerun_a0_opt` baseline, and Figure 3.2 overlays all core A0 curves.

Interpretation. The optimized A0 baseline, the non-optimized critic-update variant, and the smaller MLP variant all finish around 93% survival. This suggests that the main A0 recipe is robust once the rerun protocol is fixed. Disabling reward normalization still reaches high peak survival, but it learns more slowly and finishes lower. The old no-validation, non-optimized baseline is clearly not competitive and should not be used as the main reference for later ablations.

Table 3.2: A0-family run configurations. The reference run is `rerun_a0_opt`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from <code>rerun_a0_opt</code>
<code>rerun_a0_opt</code>	$3 \times 256 / 3 \times 256$	true	0.0	true	Reference baseline
<code>rerun_a0_nono</code>	$3 \times 256 / 3 \times 256$	true	0.0	false	Disables optimized critic updates
<code>rerun_a01</code>	$2 \times 256 / 2 \times 256$	true	0.0	true	Uses a smaller MLP actor and critic
<code>rerun_a02</code>	$3 \times 256 / 3 \times 256$	false	0.0	true	Disables reward normalization
<code>old_a0_nono</code>	$3 \times 256 / 3 \times 256$	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
<code>rerun_bias_05</code>	$3 \times 256 / 3 \times 256$	true	0.5	true	Uses action-0 bias 0.5
<code>rerun_bias_07</code>	$3 \times 256 / 3 \times 256$	true	0.7	true	Uses action-0 bias 0.7

A0 core reruns: `rerun_a0_opt` baseline pairwise comparisons

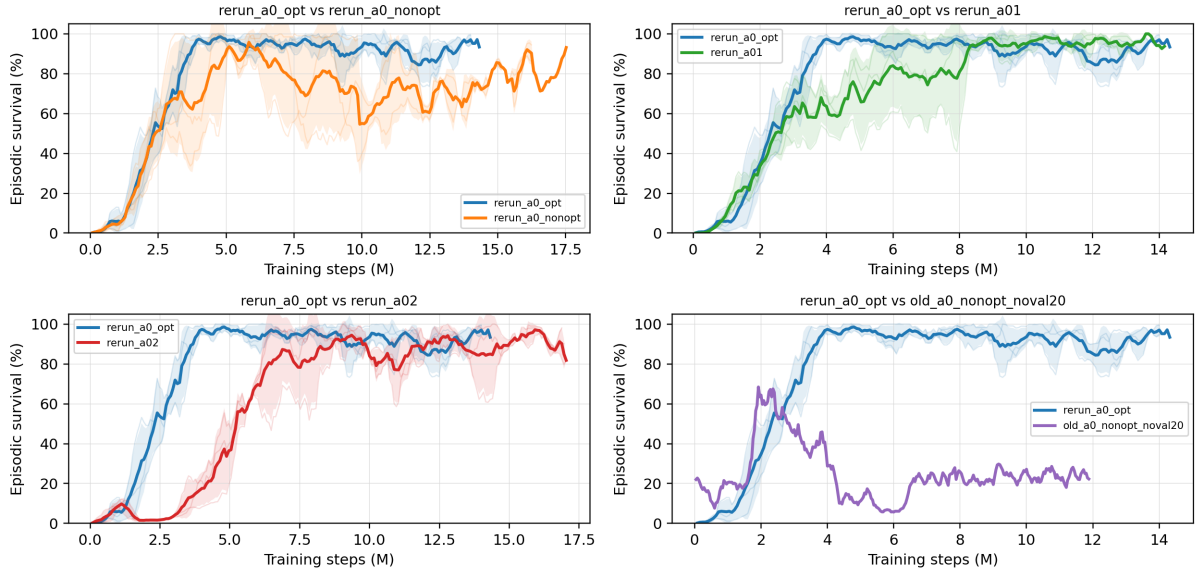


Figure 3.1: Pairwise A0 rerun comparisons against the `rerun_a0_opt` baseline.

A0 core reruns: rerun_a0_opt vs rerun_a0_nonopt / rerun_a01 / rerun_a02 / old noval20 nonopt

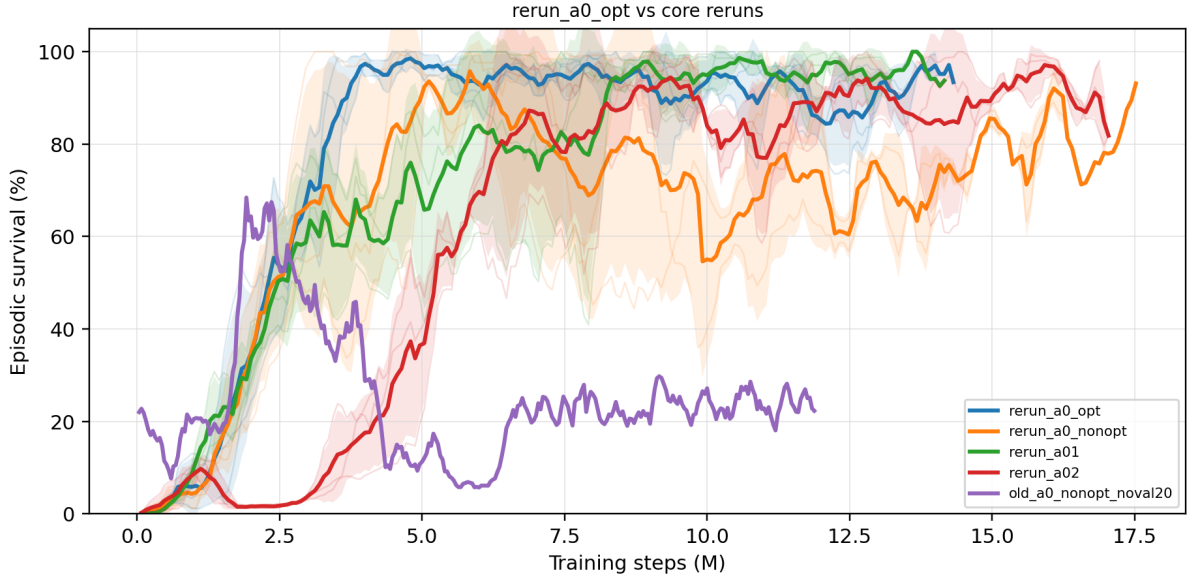


Figure 3.2: All A0 core rerun survival curves shown on one axis.

Table 3.3: A0 core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
rerun_a0_opt	A0 optimized baseline	93.4	98.6
rerun_a0_nonopt	Disable optimized critic update	93.1	95.8
rerun_a01	Smaller MLP actor/critic	93.7	100.0
rerun_a02	Disable reward normalization	81.8	97.1
old_a0_nonopt_noval20	Older non-optimized protocol	22.3	68.4

3.4.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning?

Baseline and changes. The A0 baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 3.4 and Figure 3.3 compare the zero-bias A0 baseline with rerun_bias_05 and rerun_bias_07.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is therefore not that larger bias is always better. Instead, the initialization prior appears to strongly shape exploration and should be treated as a first-order ablation variable.

Table 3.4: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
<code>rerun_a0_opt</code>	93.4	98.6
<code>rerun_bias_05</code>	50.8	70.2
<code>rerun_bias_07</code>	99.3	100.0

rerun_a0_opt baseline vs rerun_bias_05 / rerun_bias_07

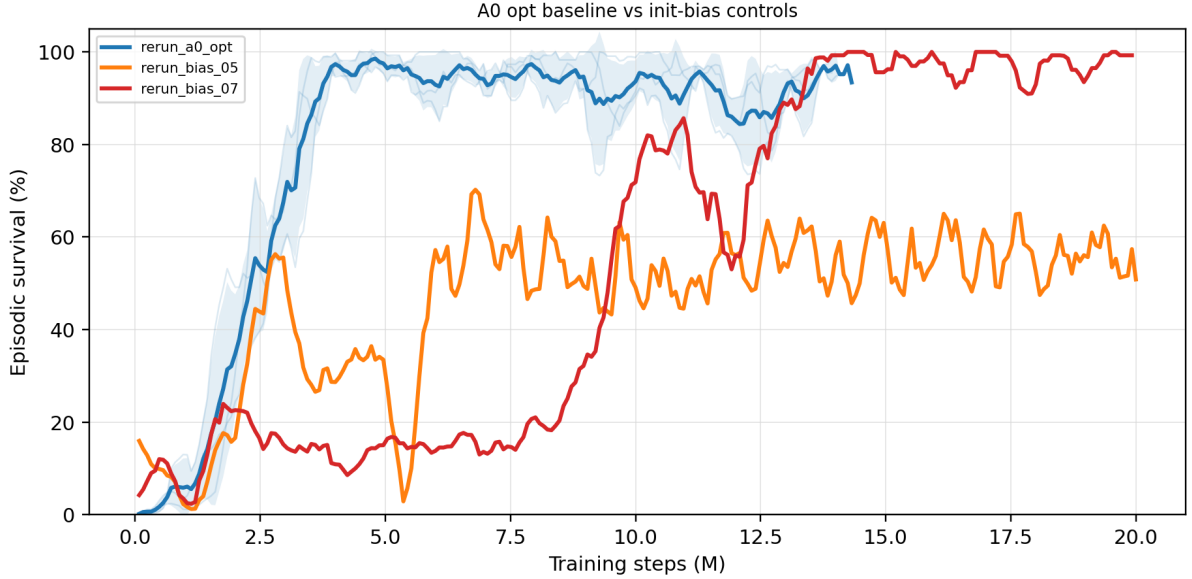


Figure 3.3: A0 baseline compared with initial do-nothing bias controls `rerun_bias_05` and `rerun_bias_07`.

3.4.3 GINE Encoder Reruns

Question. Can graph encoders improve topology-control performance, and which GINE design choices matter most?

Baseline and changes. The comparison starts from the historical `best_00` GINE baseline. The reruns then test lighter GINE encoders, MLP versus GNN critics, optimized critic updates, flat-feature concatenation, A0-style entropy, and different do-nothing-bias settings.

Protocol. All GINE runs use `gnn_type = gine`, reward normalization, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the A0 experiments. The reference run is `best_00`.

Implementation detail. The complete GINE/GAT graph-actor pipeline, including edge-conditioned message passing, pooling, and the action head, is described in Appendix A. This block separates representational changes from optimization changes. The GINE encoder changes how grid observations are embedded, while the critic type, flat-feature concatenation, entropy setting, and critic-update schedule affect how the PPO objective is optimized.

Evidence. Table 3.5 records the run configurations, Table 3.6 gives the selected survival values, and Figure 3.4 shows the pairwise survival curves against `best_00`.

Interpretation. The strongest GINE result is `best_14`, which reaches about 95.7% final survival. This suggests that the graph encoder can be competitive when it is paired with a simpler MLP critic and stable A0-style training choices. The heavier historical GINE baseline has high peak survival but much weaker final survival, which points to optimization stability rather than representational capacity as the main issue.

Table 3.5: GINE rerun configuration summary. The reference run is **best_00**.

Run	Actor/critic encoder	GNN hid- den/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from best_00
best_00	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
best_10	gnn / mlp	64 / 1	$2 \times 128 / 2 \times 128$	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
best_11	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
best_12	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
best_13	gnn / gnn	128 / 2	$3 \times 256 / 3 \times 256$	true	true	0.01	0.0	true	A0-style heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
best_14	gnn / mlp	64 / 1	$2 \times 128 / 2 \times 128$	false	false	0.01	0.0	true	Light A0-style GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Table 3.6: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
best_00	Historical GINE baseline	59.2	93.9
best_10	Light GINE with MLP critic	78.9	90.4
best_12	Optimized critic and bias 0.0	82.5	85.7
best_14	Light A0-style GINE with MLP critic	95.7	97.4

configs/gine_s0_s1_s2: best_00 vs best_10-14

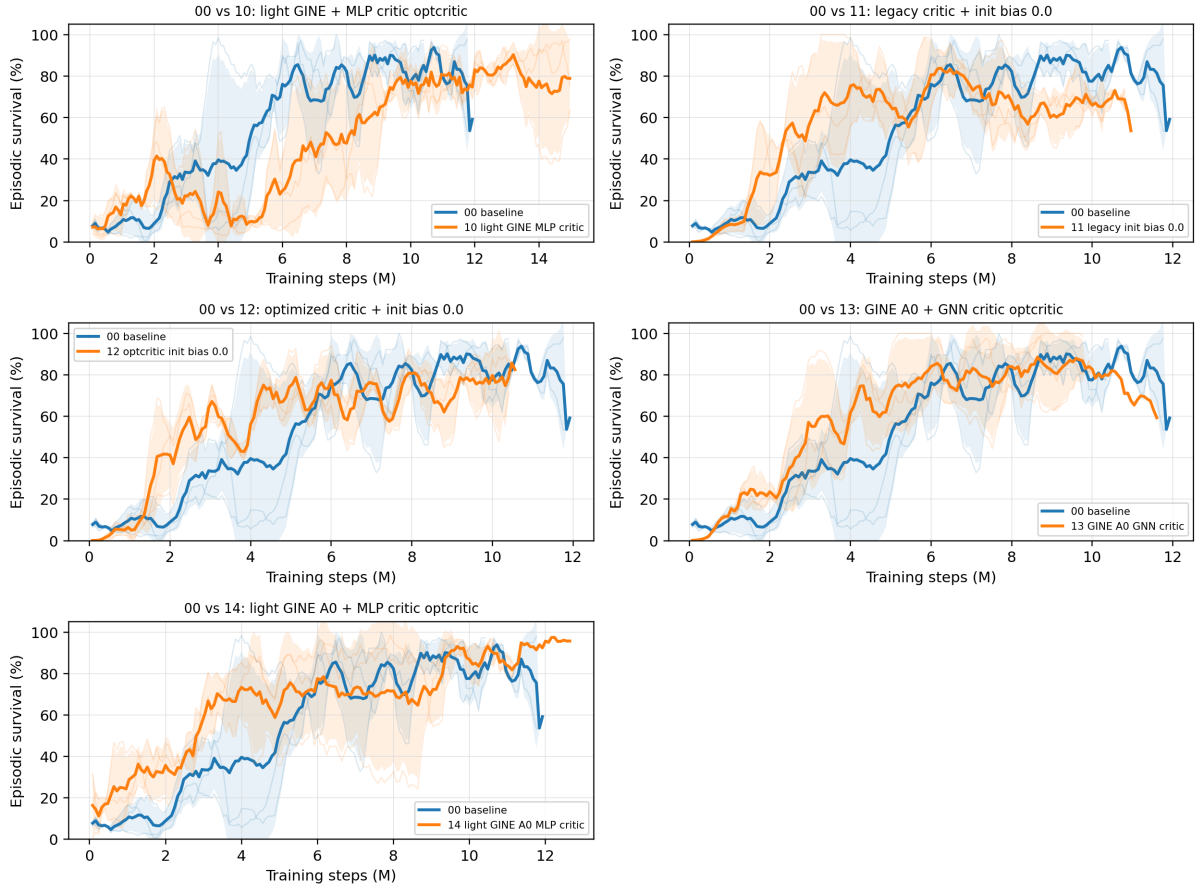


Figure 3.4: Seed-aggregated test episodic survival for GINE best_00 against best_10–best_14.

3.4.4 Heavy GINE

Question. Does the heavier concat-flat GINE baseline fail because it lacks capacity, or because specific architectural and optimization choices make it unstable?

Baseline and changes. The reference run is best_00, the historical shared-actor, concat-flat GINE with a GNN critic and legacy critic updates. The best-search 2 family compares this baseline against controlled alternatives that change the flat-feature concatenation, critic type, actor sharing, GNN size, entropy schedule, node identifiers, message-passing operator, and initialization bias.

Protocol. The configurations are read from configs/gine_s0_s1_s2. Each family is run with seeds s0, s1, and s2. Unless changed by the family, the runs use reward normalization, deterministic evaluation, 5 evaluation episodes, 15,000,000 total timesteps, evaluation every 80,000 timesteps, and the same 80/20 chronic split.

Implementation detail. Table 3.7 gives the parameter comparison used for this ablation. The table is based on the `s0` TOML file for each family; the corresponding `s1` and `s2` files keep the same configuration except for the training seed and run name.

Evidence. Figure 3.5 compares the `best_00` baseline against each heavy-GINE alternative. Each panel shows the seed-averaged test-survival curve for `best_00` and one comparison family, with faint seed curves and variability bands.

Interpretation. The broad search suggests that the heavy baseline is not limited by capacity alone. Several simpler or more constrained variants improve final survival over `best_00`, especially removing node-ID embeddings, removing flat concatenation, using an MLP critic, and combining a light GINE actor with an MLP critic. In contrast, simply switching message passing to GAT or weighted GCN, or removing the initial do-nothing bias while keeping the heavy GINE/GNN-critic structure, does not clearly solve the stability problem.

Table 3.7: Heavy-GINE best-search 2 parameter comparison from configs/gine_s0_s1_s2.

Run	Main change from best_00	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
best_000	Removes flat-observation concatenation	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	no	yes	yes	yes	0.02–0.02	0.7	no
best_00	Reference: heavy concat-flat GINE with GNN critic	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
best_01	Enables optimized critic updates	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	yes
best_02	Replaces the GNN critic with an MLP critic	gine	2×128	mlp	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
best_03	Uses non-shared actor GNN weights	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	no	yes	yes	0.02–0.02	0.7	no
best_04	Uses a lighter GINE encoder	gine	1×64	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
best_05	Decays entropy from 0.02 to 0.0	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.00	0.7	no
best_06	Removes learned node-ID embeddings	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	no	0.02–0.02	0.7	no
best_07	Replaces GINE with GAT message passing	gat	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
best_08	Replaces GINE with weighted GCN message passing	gcn	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.7	no
best_09	Light no-concat GINE with MLP critic and optimized critic updates	gine	1×64	mlp	$2 \times 128 / 2 \times 128$	no	yes	no	yes	0.02–0.02	0.7	yes
best_10	Light concat-flat GINE with MLP critic and optimized critic updates	gine	1×64	mlp	$2 \times 128 / 2 \times 128$	yes	yes	no	yes	0.02–0.02	0.7	yes
best_11	Removes the initial do-nothing bias	gine	2×128	gnn	$3 \times 256 / 3 \times 256$	yes	yes	yes	yes	0.02–0.02	0.0	no

configs/gine_s0_s1_s2: baseline 00 vs each alternative

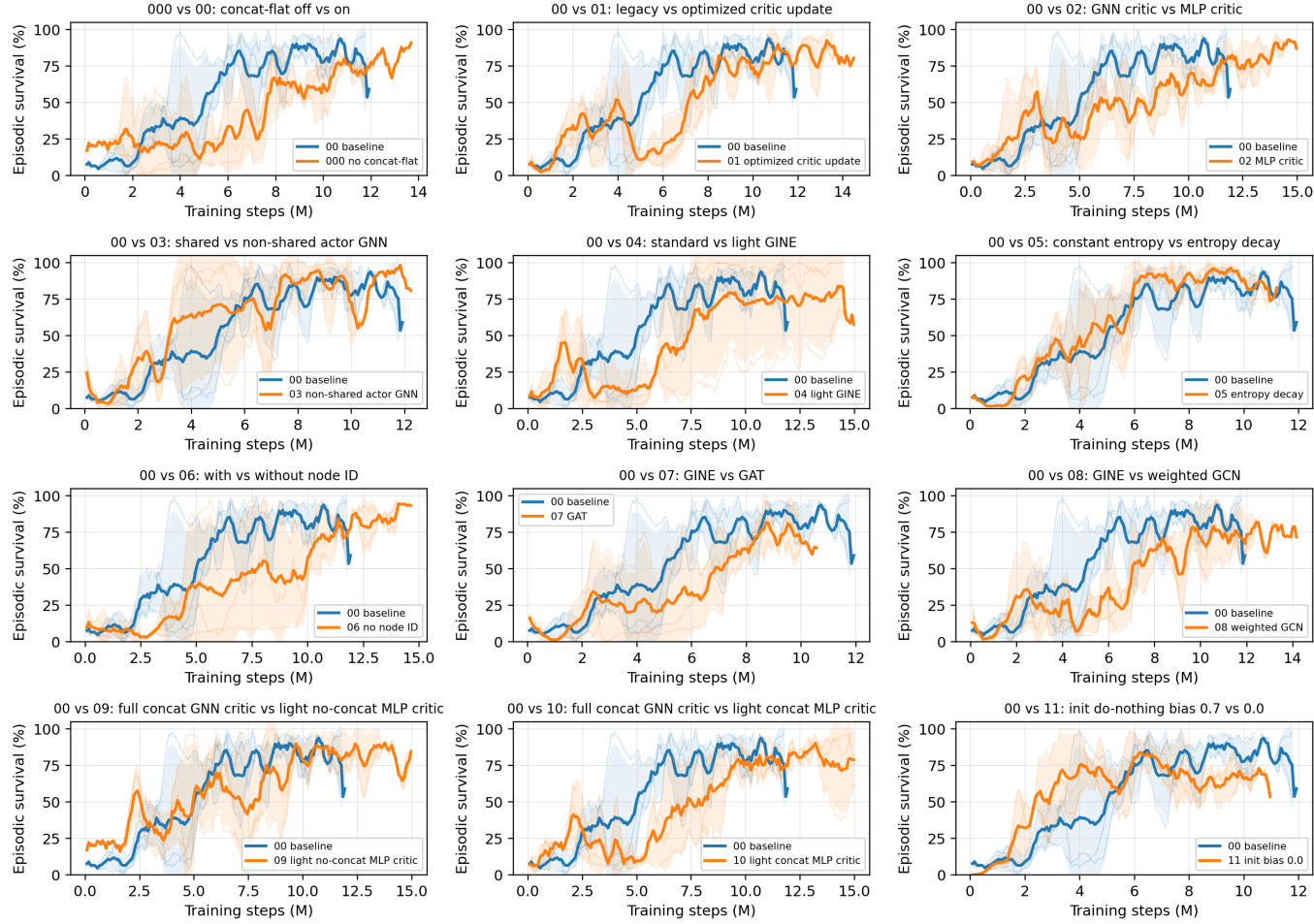


Figure 3.5: Heavy-GINE best-search 2 survival comparisons. Each subplot compares the **best_00** heavy concat-flat GINE baseline against one controlled alternative from `configs/gine_s0_s1_s2`.

3.5 WCCI Action-Space Reduction

The WCCI bus36 setting has a much larger discrete topology-action space than bus14. Before training MAPPO on this environment, the action space is reduced offline from simulated action outcomes. During rollouts, states are collected when the grid is stressed,

$$\rho_{\max} > 0.90.$$

For each collected state s , candidate actions are simulated and compared against do nothing:

$$\Delta(s, a) = \rho_{\text{after}}(s, a) - \rho_{\text{after}}(s, 0).$$

An action is counted as useful when it lowers the post-action loading relative to do nothing, i.e. $\Delta(s, a) < -\epsilon$. Each action is then scored by its empirical improvement rate,

$$\text{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

with a minimum-count filter to avoid selecting rarely sampled actions. The final reduced action space keeps the best actions per agent, while always preserving action 0. The candidate sets are then sanity-checked with a one-step simulator-greedy controller: at each risky state, the controller simulates the available reduced actions and executes the best improving one.

Table 3.8: One-step greedy evaluation of WCCI reduced action spaces on 50 random test chronics. All rows use the same chronic sample (`chronic_sample_seed=0`); the do-nothing mean survival on this sample is 28.30%.

Reduced action space	Kept actions per agent (a_0, a_1, a_2, a_3)	Greedy survival (%)	Do-nothing survival (%)	Survival delta (%)	Greedy full survival (%)
Do-nothing reference	1, 1, 1, 1	–	28.30	–	–
WCCI $h = 1$, mk64	64, 64, 64, 64	55.24	28.30	26.94	22.00
WCCI $h = 1$, mk128	77, 128, 127, 128	60.65	28.30	32.35	22.00
WCCI $h = 1$, mk256	77, 256, 127, 256	68.61	28.30	40.31	36.00
WCCI $h = 1$, mk512	77, 512, 127, 512	71.21	28.30	42.90	40.00
WCCI $h = 1$, mk1024	77, 1024, 127, 1024	78.18	28.30	49.87	54.00

3.6 Decentralized Sparse Intervention Control

Question. Can the agents keep high survival while behaving more like operators: acting rarely, acting locally, acting mainly when the grid state makes an intervention useful, and keeping the final policy decentralized?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i , the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \quad a_i \in \{0, 1, \dots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{\text{eval}} = \arg \max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests four ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, and adaptive Lagrangian intervention budgets.

Motivation. The intervention gate is motivated by hierarchical topology-control work, where the action can be decomposed into “do nothing versus intervene”, then where to act, then which local topology configuration to apply [3], [4]. In this thesis only the first level is adapted: each regional agent independently decides whether to do nothing or to execute one of its existing local topology actions. The fixed and adaptive sparse-control objectives are closer to constrained and budgeted RL: non-idle topology interventions are treated as a limited resource, following the general Lagrangian view of constrained policy optimization and budgeted control [5], [6], [7]. This is also operationally meaningful because switching frequency and topology deviation are real power-grid objectives, not just cosmetic metrics [8].

3.6.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise,} \end{cases} \quad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

3.6.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i \mid o_i), \quad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i \mid o_i), \quad b_i \in \{1, \dots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i \mid g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \quad P(a_i = a > 0) = P(g_i = 1)P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1)H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate}H(g_i) + \alpha_{nonidle}H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

3.6.3 Fixed Sparse-Action Penalty

Implementation detail. The Sparse16 sweep adds a fixed reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed} \mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \quad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \quad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. All Sparse16 runs set `safe_intervention_penalty = 0.0`, so the penalty is not rho-weighted.

Interpretation. Sparse16 changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. The key comparison is `sparse16_flat_p010` versus `sparse16_gated_p010`, which asks whether the gate adds value once a simple sparse objective already exists.

3.6.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent,

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0] w_i(s_t),$$

and the desired constraint is

$$\mathbb{E}_{\pi_i} [c_i(s_t, a_{i,t})] \leq d.$$

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \text{clip} \left(\lambda_i + \eta (\hat{C}_i - d), 0, \lambda_{\max} \right), \quad \hat{C}_i = \frac{1}{T} \sum_{t=1}^T c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d , future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Three cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Global-safe cost:** $w_i^{global}(s_t) = \sigma(k(\rho_{safe} - \rho_t^{global}))$. This penalizes non-idle actions mostly when the whole grid is safe.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The first AIB folder, `configs/adaptive_intervention_budget_7`, contains the main local-budget ablations:

- `aib_00_flat_local_t020`: flat actor, local-safe cost, $d = 0.20$;
- `aib_01_flat_local_t010`: stricter local-safe budget, $d = 0.10$;
- `aib_02_flat_local_t035`: looser local-safe budget, $d = 0.35$;
- `aib_03_gate_hgreedy_sep_local_t020`: gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- `aib_04_flat_nonidle_t020`: flat actor with adaptive plain non-idle cost, $d = 0.20$.

The comparison `aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020` isolates the value of local rho weighting.

The mechanism-ablation folder `configs/adaptive_intervention_budget_mechanism_15` contains 15 additional flat-actor runs. It separates fixed versus adaptive coefficients, plain non-idle versus state-dependent costs, global versus local rho weighting, and rho-threshold sensitivity.

Table 3.9: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Sparse16	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design

Table 3.10: Main sparse-control comparisons to report.

Question	Comparison
Does a fixed sparse penalty already work?	Baseline versus <code>sparse16_flat_p010</code>
Does the gate help beyond a sparse objective?	<code>sparse16_flat_p010</code> versus <code>sparse16_gated_p010</code>
Fixed versus adaptive coefficient?	<code>sparse16_flat_p010</code> or <code>aibm_00_fixed_nonidle_p006</code> versus <code>aib_04_flat_nonidle_t020</code>
Does local-safe weighting matter?	<code>aib_00_flat_local_t020</code> versus <code>aib_04_flat_nonidle_t020</code>
Does global-safe weighting matter?	<code>aibm_02_adaptive_global_t020</code> versus <code>aib_00_flat_local_t020</code>
Is $\rho_{safe} = 0.90$ special?	<code>aibm_03</code> with 0.85, <code>aib_00</code> with 0.90, and <code>aibm_04</code> with 0.95
Is the heuristic only an evaluation trick?	Rho-heuristic runs versus learned sparse/AIB runs

Full-test checkpoint summaries. The following tables report the average full-test episodic survival over the three evaluated seeds. The survival values are computed directly from the JSON files in `Topology_Task/outputs/full_test_eval`. These JSON files also record where the compact action-distribution artifacts were written. The action-0 fractions are computed from the corresponding local `Topology_Task/outputs/full_test_eval_actions/*/action_summary.json` files and averaged over

seeds. The all-agent action-0 column is the arithmetic mean of the three per-agent action-0 fractions. The do-nothing row is a reference policy that always selects action 0; its survival is computed from `Topology_Task/outputs/do_nothing_eval/bus14_test_20`. Remaining “–” entries indicate that the compact action summaries for those full-test jobs are not available locally.

Table 3.11: Full-test summary for evaluation-time rho heuristics.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Global rho heuristic, $\rho_{safe} = 0.90$	98.36	0.936	0.933	0.936	0.940
Local rho heuristic, $\rho_{safe} = 0.90$	97.07	0.968	0.992	0.985	0.928

Table 3.12: Full-test summary for intervention-gated actors.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Intervention gate, final-action MAP	89.82	0.802	0.826	0.889	0.690
Intervention gate, hierarchical greedy	93.66	0.395	0.066	0.844	0.276

Table 3.13: Full-test summary for Sparse16 flat-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat Sparse16, $\lambda_{fixed} = 0.000$	98.23	0.494	0.524	0.463	0.496
Flat Sparse16, $\lambda_{fixed} = 0.001$	96.75	0.486	0.642	0.401	0.416
Flat Sparse16, $\lambda_{fixed} = 0.003$	93.97	0.575	0.691	0.346	0.688
Flat Sparse16, $\lambda_{fixed} = 0.010$	97.44	0.905	0.907	0.954	0.853

Metrics to report. The main performance metric remains episodic survival. It should be paired with sparsity and physical-diagnostic metrics: per-agent action-0 fractions, per-agent non-idle rates, the distribution of how many agents act simultaneously, intervention-budget multipliers, budget costs and violations, the rate of interventions in costly safe states, pre- and post-action max rho, and topology-distance changes.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, `sparse16_flat_p010` is a strong fixed-penalty baseline, while the most principled learned sparse-control candidate is `aib_00_flat_local_t020`: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight.

3.7 Current Takeaway

The current reruns indicate that the strongest direction is not a single isolated modification. Stable performance comes from a combination of A0-style training choices, reward normalization, careful initialization, and architectures that keep the critic and graph encoder simple enough to optimize reliably. Optimized critic updates and lighter GINE/MLP-critic variants can improve the final GINE rerun performance, but the A0 baseline remains the clearest stable reference point for future ablations. The sparse-control roadmap adds a second axis to the thesis: after survival is stabilized, the next question is whether the same decentralized agents can achieve high survival with fewer unnecessary topology interventions.

Table 3.14: Full-test summary for Sparse16 gated-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Gated Sparse16, $\lambda_{fixed} = 0.000$	78.51	0.787	0.747	0.949	0.665
Gated Sparse16, $\lambda_{fixed} = 0.001$	73.82	0.862	0.956	0.832	0.799
Gated Sparse16, $\lambda_{fixed} = 0.003$	94.59	0.887	0.934	0.929	0.799
Gated Sparse16, $\lambda_{fixed} = 0.010$	85.69	0.974	0.990	0.984	0.947

Table 3.15: Full-test summary for adaptive intervention budget runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat local-safe AIB, $d = 0.20$	98.31	0.978	0.980	0.989	0.966
Flat local-safe AIB, $d = 0.10$	98.39	0.979	0.992	0.995	0.949
Flat local-safe AIB, $d = 0.35$	96.61	0.934	0.959	0.947	0.896
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	33.87	0.999	1.000	1.000	0.998
Flat non-idle AIB, $d = 0.20$	95.08	0.986	0.997	0.992	0.970

Table 3.16: Full-test summary for the teacher-student distillation runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Teacher: MAPPO with local rho heuristic	97.07	0.968	0.992	0.985	0.928
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	92.14	0.939	0.974	0.925	0.919
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	90.76	0.957	0.982	0.948	0.940

Appendix A

GINE and GAT Graph-Actor Architectures

This appendix specifies the complete edge-aware actor used in the graph experiments. GINE and GAT receive the same graph inputs and use the same pooling and action head. Their only architectural difference is the message-passing operator: GINE inserts an edge embedding into the message content, whereas GAT uses an edge embedding to decide how strongly the message is weighted.

A.1 Shared input and encoder pipeline

For agent a , the encoder receives the local graph $\mathcal{G}_t^{(a)}$. Candidate edges with `edge_mask`=0 are removed before message passing, so the layer sees only the active directed set $\mathcal{E}_t^{(a)}$. The source of $u \rightarrow v$ sends a message and v aggregates it. Node and edge masks do not become learned features.

Each raw node vector $\mathbf{x}_v$ first passes through

$$\mathbf{h}_v^{(0)} = \text{LayerNorm}(\text{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \quad (\text{A.1})$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE or GAT layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \text{LayerNorm}(\text{ReLU}(\tilde{\mathbf{h}}_v^{(k+1)})). \quad (\text{A.2})$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

A.2 GINE message passing

For an active edge $u \rightarrow v$, layer k projects the complete edge feature vector and adds it to the sender state:

$$\tilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \quad (\text{A.3})$$

$$\mathbf{m}_{u \rightarrow v}^{(k)} = \text{ReLU}(\mathbf{h}_u^{(k)} + \tilde{\mathbf{e}}_{uv}^{(k)}), \quad (\text{A.4})$$

$$\tilde{\mathbf{h}}_v^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon) \mathbf{h}_v^{(k)} + \sum_{u: (u,v) \in \mathcal{E}_t} \mathbf{m}_{u \rightarrow v}^{(k)} \right). \quad (\text{A.5})$$

The implementation uses the PyTorch Geometric default $\epsilon = 0$ without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation A.5 shows why edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise,

reversing a typed heterogeneous edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

A.3 GAT message passing

GAT first computes a score for every active incoming edge and attention head. For head r , the implementation has the form

$$s_{uv}^{(k,r)} = \text{LeakyReLU}\left(\mathbf{a}_{s,r}^\top \mathbf{W}_{s,r} \mathbf{h}_u^{(k)} + \mathbf{a}_{d,r}^\top \mathbf{W}_{d,r} \mathbf{h}_v^{(k)} + \mathbf{a}_{e,r}^\top \mathbf{W}_{e,r} \mathbf{e}_{uv}\right), \quad (\text{A.6})$$

$$\alpha_{uv}^{(k,r)} = \frac{\exp s_{uv}^{(k,r)}}{\sum_{w:(w,v) \in \mathcal{E}_t} \exp s_{vw}^{(k,r)}}, \quad (\text{A.7})$$

$$\tilde{\mathbf{h}}_v^{(k+1)} = \frac{1}{H} \sum_{r=1}^H \sum_{u:(u,v) \in \mathcal{E}_t} \alpha_{uv}^{(k,r)} \mathbf{W}_r \mathbf{h}_u^{(k)}. \quad (\text{A.8})$$

GAT uses $H = 4$ heads and `concat=false`, so the four head outputs are averaged and the layer still returns 128 features. The edge embedding affects α_{uv} : it changes how much attention the receiver assigns to the sender. In the standard GAT operator used here, the edge vector is not added to the value vector $\mathbf{W}_r \mathbf{h}_u$. This is the central difference from GINE.

GAT internally adds a self-loop for every node. That computational self-loop is separate from the optional explicit `relation_self` edge in the graph schema.

Table A.1: Exact architectural difference between the implemented GINE and GAT actors.

Component	GINE	GAT
Edge use	Added to the sender state inside every message	Enters the attention score for every incoming edge
Neighbor aggregation	Unweighted sum	Softmax-weighted sum
Self information	Explicit $(1 + \epsilon) \mathbf{h}_v$ term	Internal GAT self-loop
Multiple heads	Not used	Four heads, averaged rather than concatenated
Post-layer operation	ReLU, then LayerNorm	ReLU, then LayerNorm
Pooling and action head	Shared implementation	Shared implementation

A.4 Pooling and graph output

With mean pooling, the graph representation after the second message-passing layer is

$$\bar{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \quad (\text{A.9})$$

where m_v is the valid-node mask. This is one mean over all valid node types. It is not a separate mean per type. Therefore:

- the busbar graph pools all busbars;
- the direct heterogeneous graph pools busbars, generators, and loads;
- the explicit-line graph also pools transmission-line nodes.

For a local actor graph, one-hop contextual nodes are included in the mean. The ordinary GINE/GAT mean readout does not restrict pooling to controlled nodes. Sum and maximum pooling change only Equation A.9; they do not change message passing.

The pooled 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \text{ReLU}(\mathbf{W}_o \bar{\mathbf{h}}_G + \mathbf{b}_o), \quad \mathbf{z}_a \in \mathbb{R}^{128}. \quad (\text{A.10})$$

When virtual-node readout is selected, the final virtual-node state replaces $\bar{\mathbf{h}}_G$; the output projection and its dimension stay the same.

A.5 Agent-specific action head

The flat observation is not concatenated to $\mathbf{z}_a$. Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \text{ReLU}(\mathbf{W}_a^{(1)}\mathbf{z}_a + \mathbf{b}_a^{(1)}), \quad (\text{A.11})$$

$$\mathbf{q}_a^{(2)} = \text{ReLU}(\mathbf{W}_a^{(2)}\mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \quad (\text{A.12})$$

$$\ell_a = \mathbf{W}_a^{(3)}\mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \quad \ell_a \in \mathbb{R}^{|\mathcal{A}_a|}, \quad (\text{A.13})$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \text{Categorical}(\text{logits} = \ell_a). \quad (\text{A.14})$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects $\arg \max_j \ell_{a,j}$. Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths $|\mathcal{A}_a|$. The critic is a separate MLP over the centralized flat state and does not reuse the actor GINE/GAT embedding.

A.6 Exact node inputs by representation

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE and GAT through the `is_*` columns.

Table A.2: Node types and nonzero input features received by GINE and GAT.

Representation	Node type	Populated input features
bus	Busbar	<code>gen_p</code> , <code>gen_theta</code> , <code>load_p</code> , <code>load_theta</code> , <code>time_before_cooldown_sub</code> , <code>domain_mask</code>
heterogeneous	Busbar	<code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>connected</code> , <code>domain_mask</code>
	Generator	<code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Load	<code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>connected</code> , <code>domain_mask</code>
heterogeneous_line	Busbar	<code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>connected</code> , <code>domain_mask</code>
	Generator	<code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Load	<code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>connected</code> , <code>domain_mask</code>
	Transmission line	<code>is_transmission_line</code> , <code>line_status</code> , <code>rho</code> , <code>timestep_overflow</code> , <code>time_before_cooldown_line</code> , optional maintenance times, <code>connected</code> , <code>domain_mask</code>
Any augmented schema	Substation summary	Learned substation-indexed vector; no raw physical feature row
	Virtual node	One learned vector shared across graphs; no raw physical feature row

A.7 Exact edge inputs by representation

The edge vector contains all edge information consumed by GINE and GAT. Inactive candidates are filtered out before the vectors in Table A.3 reach a convolution.

Table A.3: Directed edge relations and edge-feature vectors received by GINE and GAT. “Relation” denotes one active relation-one-hot column.

Representation	Directed relation	Edge features
bus	Physical busbar $\leftrightarrow$ busbar	line_status , rho , timestep_overflow , time_before_cooldown_line , optional maintenance times; no relation one-hot
	Optional self or same-substation	Same width; physical values zero and rho = 1; no relation one-hot
heterogeneous	Physical busbar $\leftrightarrow$ busbar	Complete measured physical-line block plus relation_physical_line
	Generator $\leftrightarrow$ busbar	Neutral physical block plus direction-specific generator relation
	Load $\leftrightarrow$ busbar	Neutral physical block plus direction-specific load relation
	Optional self or same-substation	Neutral physical block plus the corresponding relation
heterogeneous_line	Busbar $\leftrightarrow$ line node	Measured rho plus an origin/extremity-and-direction relation
	Generator $\leftrightarrow$ busbar	rho = 1 plus direction-specific generator relation
	Load $\leftrightarrow$ busbar	rho = 1 plus direction-specific load relation
	Optional self or same-substation	rho = 1 plus the corresponding relation
Any augmented schema	Busbar $\rightarrow$ summary/virtual, with optional reverse	Zero edge vector except neutral rho = 1 when present; no dedicated relation one-hot in the ordinary GINE/GAT encoder

The complete relation names and activation rules are given in Tables 2.5 and 2.7. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to both edge-aware models: GINE changes message content and GAT changes attention.

Appendix B

Full-Test Checkpoint Registry

Table [B.1](#) reports the checkpoint step used for each full-test result table. The values are the `checkpoint_global_step` fields stored in the corresponding JSON files under `Topology_Task/outputs/full_test_eval`. The do-nothing policy has no learned checkpoint. For the behavioral cloning students, the stored step is the student checkpoint’s optimizer-step count rather than a MAPPO environment step.

Table B.1: Checkpoint steps used for the full-test evaluations.

Run type	Seed 0 checkpoint step	Seed 1 checkpoint step	Seed 2 checkpoint step
Do-nothing policy	—	—	—
Baseline flat policy	13,600,000	13,040,000	13,760,000
Global rho heuristic, $\rho_{safe} = 0.90$	13,920,000	14,320,000	14,800,000
Local rho heuristic, $\rho_{safe} = 0.90$	14,160,000	14,400,000	14,160,000
Intervention gate, final-action MAP	12,000,000	12,720,000	13,760,000
Intervention gate, hierarchical greedy	7,120,000	14,720,000	13,760,000
Flat Sparse16, $\lambda_{fixed} = 0.000$	9,600,000	12,160,000	7,920,000
Flat Sparse16, $\lambda_{fixed} = 0.001$	10,640,000	11,520,000	11,600,000
Flat Sparse16, $\lambda_{fixed} = 0.003$	12,000,000	12,960,000	12,480,000
Flat Sparse16, $\lambda_{fixed} = 0.010$	8,960,000	5,760,000	12,560,000
Gated Sparse16, $\lambda_{fixed} = 0.000$	8,480,000	4,320,000	6,000,000
Gated Sparse16, $\lambda_{fixed} = 0.001$	5,760,000	7,600,000	3,280,000
Gated Sparse16, $\lambda_{fixed} = 0.003$	8,960,000	7,120,000	7,600,000
Gated Sparse16, $\lambda_{fixed} = 0.010$	12,000,000	7,840,000	7,920,000
Flat local-safe AIB, $d = 0.20$	13,120,000	9,360,000	12,160,000
Flat local-safe AIB, $d = 0.10$	12,880,000	13,200,000	12,720,000
Flat local-safe AIB, $d = 0.35$	12,800,000	5,360,000	11,520,000
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	14,960,000	5,280,000	16,800,000
Flat non-idle AIB, $d = 0.20$	12,480,000	14,880,000	11,920,000
Teacher: MAPPO with local rho heuristic	14,160,000	14,400,000	14,160,000
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	12,375	14,994	13,617
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	12,375	14,994	13,617

Bibliography

- [1] B. Donnot, “Grid2op: A testbed platform to model sequential decision making in power systems,” *arXiv preprint arXiv:2011.07929*, 2020.
- [2] C. Yu et al., “The surprising effectiveness of ppo in cooperative multi-agent games,” *Advances in Neural Information Processing Systems*, 2022.
- [3] J. Manczak et al., *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: [2311.02129](https://arxiv.org/abs/2311.02129). [Online]. Available: <https://arxiv.org/abs/2311.02129>
- [4] E. van der Sar et al., *Hierarchical multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: [2310.02605](https://arxiv.org/abs/2310.02605). [Online]. Available: <https://arxiv.org/abs/2310.02605>
- [5] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” *International Conference on Machine Learning*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [6] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID lagrangian methods,” *International Conference on Machine Learning*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.03964>
- [7] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, “Budgeted reinforcement learning in continuous state space,” *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01004>
- [8] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: [2502.00040](https://arxiv.org/abs/2502.00040). [Online]. Available: <https://arxiv.org/abs/2502.00040>
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [10] A. Marot et al., “Learning to run a power network challenge for training topology controllers,” *Electric Power Systems Research*, 2021.
- [11] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *International Conference on Learning Representations*, 2017.